Cosmos SDK Module System: Complete Guide

목차

- 개요
- Module이란?
- AppModule 인터페이스
- Module Manager
- Module 생명주기
- Keeper 패턴
- Inter-Module Communication
- 실전: Custom Module 개발
- Best Practices
- 참고 자료

개요

Cosmos SDK의 Module System은 블록체인 애플리케이션을 모듈화된 컴포넌트로 분리하는 아키텍처 패턴입니다. 각 모듈은 독립적인 기능을 담당하며, 서로 조합하여 완전한 블록체인 애플리케이션을 구성합니다.

학습 목표:

- Module의 개념과 역할 이해
- AppModule 인터페이스 완전 파악
- ModuleManager의 작동 방식
- Keeper 패턴을 통한 상태 관리
- 모듈 간 통신 메커니즘

예상 학습 시간: 4-5시간

Module이란?

모듈의 정의

모듈은 특정 기능을 구현하는 독립적인 코드 단위입니다. Cosmos SDK는 여러 내장 모듈을 제공하며, 개발자는 커스텀 모듈을 추가할 수 있습니다.

주요 내장 모듈

1. auth - 인증

- Account 관리
- Account number, sequence 관리
- Signature 검증 지원

2. bank - 토큰 전송

- 코인 전송 기능
- 잔액 조회
- Supply 관리

3. staking - 위임 증명

- Validator 관리
- Delegation, Undelegation
- Slashing 연동

4. distribution - 리워드 분배

- 블록 리워드 분배
- Commission 관리
- Withdrawal 처리

5. gov - 거버넌스

- Proposal 생성 및 투표
- Parameter 변경
- Software upgrade 관리

6. slashing - 처벌

- Downtime tracking
- Byzantine behavior 처벌
- Validator jailing

7. mint - 토큰 발행

- 인플레이션 관리
- 블록 리워드 계산

8. params - **파라미터 관리**

- 모듈 파라미터 저장
- 거버넌스를 통한 변경

모듈의 구조

일반적인 모듈 디렉터리 구조:

```
x/mymodule/
├── client/              # CLI 및 REST 클라이언트
│   ├── cli/
│   │   ├── query.go     # 쿼리 명령어
│   │   └── tx.go        # 트랜잭션 명령어
│   └── rest/            # (레거시) REST endpoints
├── keeper/              # 상태 관리 로직
│   ├── keeper.go        # Keeper 구조체
│   ├── msg_server.go    # 메시지 핸들러
│   ├── query_server.go  # 쿼리 핸들러
│   └── ...
├── types/               # 타입 정의
│   ├── codec.go         # 인코딩/디코딩
│   ├── events.go        # 이벤트 타입
│   ├── expected_keepers.go  # 다른 모듈의 Keeper 인터페이스
│   ├── genesis.go       # Genesis 상태
│   ├── keys.go          # Store key 정의
│   ├── msgs.go          # 메시지 타입
│   ├── params.go        # 파라미터 정의
│   ├── query.pb.go      # gRPC 쿼리 (생성됨)
│   ├── tx.pb.go         # gRPC 메시지 (생성됨)
│   └── ...
├── genesis.go           # Genesis 로직
├── module.go            # AppModule 인터페이스 구현
└── proto/
    └── mymodule/
        ├── v1/
        │   ├── query.proto
        │   ├── tx.proto
        │   └── types.proto
        └── ...
```

AppModule 인터페이스

AppModule은 모듈이 애플리케이션과 상호작용하기 위한 인터페이스입니다.

AppModuleBasic

기본 모듈 정보를 제공하는 인터페이스:

```go
// types/module/module.go
type AppModuleBasic interface {
    // 모듈 이름
    Name() string

    // Genesis 기본값 반환
    DefaultGenesis(cdc codec.JSONCodec) json.RawMessage

    // Genesis 상태 검증
    ValidateGenesis(cdc codec.JSONCodec, config client.TxEncodingConfig, bz json.RawMessage) error

    // REST routes 등록 (deprecated)
    RegisterRESTRoutes(clientCtx client.Context, rtr *mux.Router)

    // gRPC Gateway routes 등록
    RegisterGRPCGatewayRoutes(clientCtx client.Context, mux *runtime.ServeMux)

    // CLI 명령어 반환
    GetTxCmd() *cobra.Command
    GetQueryCmd() *cobra.Command

    // Protobuf 타입 등록
    RegisterInterfaces(registry codectypes.InterfaceRegistry)
}
```

구현 예시:

```go
// x/mymodule/module.go
type AppModuleBasic struct {
    cdc codec.Codec
}

func (AppModuleBasic) Name() string {
    return types.ModuleName  // "mymodule"
}

func (AppModuleBasic) RegisterInterfaces(registry codectypes.InterfaceRegistry) {
    types.RegisterInterfaces(registry)
}

func (AppModuleBasic) DefaultGenesis(cdc codec.JSONCodec) json.RawMessage {
    return cdc.MustMarshalJSON(types.DefaultGenesisState())
}

func (AppModuleBasic) ValidateGenesis(cdc codec.JSONCodec, config client.TxEncodingConfig, bz json.RawMessage) error {
    var data types.GenesisState
    if err := cdc.UnmarshalJSON(bz, &data); err != nil {
        return fmt.Errorf("failed to unmarshal genesis state: %w", err)
    }
    return data.Validate()
}

func (AppModuleBasic) RegisterGRPCGatewayRoutes(clientCtx client.Context, mux *runtime.ServeMux) {
    types.RegisterQueryHandlerClient(context.Background(), mux, types.NewQueryClient(clientCtx))
}

func (AppModuleBasic) GetTxCmd() *cobra.Command {
    return cli.GetTxCmd()
}

func (AppModuleBasic) GetQueryCmd() *cobra.Command {
    return cli.GetQueryCmd()
}
```

AppModule

런타임 기능을 제공하는 인터페이스:

```go
// Cosmos SDK v0.50+ (최신)
// Deprecated: 기존 AppModule은 extension interfaces로 대체됨
type AppModule interface {
    appmodule.AppModule  // Core AppModule 임베드
    AppModuleBasic
}

// Core AppModule (cosmossdk.io/core/appmodule)
type AppModule interface {
    IsAppModule()        // 마커 메소드
    IsOnePerModuleType() // depinject용 마커
}
```

Extension Interfaces (v0.50+)

Cosmos SDK v0.50+에서는 **모듈 기능을 작은 인터페이스로 분리**했습니다:

```go
// 1. Services 등록
type HasServices interface {
    RegisterServices(Configurator)
}

// 2. Genesis 처리
type HasGenesis interface {
    DefaultGenesis(codec.JSONCodec) json.RawMessage
    ValidateGenesis(codec.JSONCodec, client.TxEncodingConfig, json.RawMessage) error
    InitGenesis(sdk.Context, codec.JSONCodec, json.RawMessage)
    ExportGenesis(sdk.Context, codec.JSONCodec) json.RawMessage
}

// 또는 Validator 업데이트가 필요한 경우
type HasABCIGenesis interface {
    InitGenesis(sdk.Context, codec.JSONCodec, json.RawMessage) []abci.ValidatorUpdate
    ExportGenesis(sdk.Context, codec.JSONCodec) json.RawMessage
}

// 3. PreBlock (v0.50+, 합의 파라미터 수정 등)
type HasPreBlocker interface {
    PreBlock(context.Context) error
}

// 4. BeginBlock (context.Context 사용!)
type HasBeginBlocker interface {
    BeginBlock(context.Context) error
}

// 5. EndBlock (context.Context 사용!)
type HasEndBlocker interface {
    EndBlock(context.Context) ([]abci.ValidatorUpdate, error)
}

// 6. Invariants (불변성 검사)
type HasInvariants interface {
    RegisterInvariants(sdk.InvariantRegistry)
}

// 7. Consensus Version (모듈 업그레이드)
type HasConsensusVersion interface {
    ConsensusVersion() uint64
}
```

주요 변경사항 (v0.47 → v0.50):

- ✅ `sdk.Context` → `context.Context` (모든 blocker 메소드)
- ✅ `abci.RequestBeginBlock` 제거 → 파라미터 없음
- ✅ `abci.RequestEndBlock` 제거 → 파라미터 없음
- ✅ 모놀리식 AppModule → 작은 Extension interfaces
- ✅ PreBlock 추가 (합의 파라미터 동적 수정 가능)

구현 예시 (v0.50+):

```go
// x/mymodule/module.go (Cosmos SDK v0.50+)
type AppModule struct {
    AppModuleBasic
    keeper keeper.Keeper
}

// Marker methods (appmodule.AppModule)
func (AppModule) IsAppModule() {}
func (AppModule) IsOnePerModuleType() {}

// HasServices: gRPC 서비스 등록
func (am AppModule) RegisterServices(cfg module.Configurator) {
    // gRPC message service 등록
    types.RegisterMsgServer(cfg.MsgServer(), keeper.NewMsgServerImpl(am.keeper))

    // gRPC query service 등록
    types.RegisterQueryServer(cfg.QueryServer(), am.keeper)

    // Migration 등록 (업그레이드 시)
    m := keeper.NewMigrator(am.keeper)
    if err := cfg.RegisterMigration(types.ModuleName, 1, m.Migrate1to2); err != nil {
        panic(err)
    }
}

// HasGenesis: Genesis 처리
func (am AppModule) InitGenesis(ctx sdk.Context, cdc codec.JSONCodec, data json.RawMessage) {
    var genesisState types.GenesisState
    cdc.MustUnmarshalJSON(data, &genesisState)
    am.keeper.InitGenesis(ctx, genesisState)
}

func (am AppModule) ExportGenesis(ctx sdk.Context, cdc codec.JSONCodec) json.RawMessage {
    gs := am.keeper.ExportGenesis(ctx)
    return cdc.MustMarshalJSON(gs)
}

// HasBeginBlocker: 블록 시작 시 실행 (v0.50+: context.Context 사용!)
func (am AppModule) BeginBlock(ctx context.Context) error {
    // SDK Context 가져오기
    sdkCtx := sdk.UnwrapSDKContext(ctx)

    // 블록 시작 시 실행할 로직
    // 예: 시간 기반 작업, 통계 업데이트 등
    return am.keeper.BeginBlocker(sdkCtx)
}

// HasEndBlocker: 블록 종료 시 실행 (v0.50+: context.Context 사용!)
func (am AppModule) EndBlock(ctx context.Context) ([]abci.ValidatorUpdate, error) {
    sdkCtx := sdk.UnwrapSDKContext(ctx)

    // 블록 종료 시 실행할 로직
    // 예: Validator set 업데이트, 리워드 분배 등
    return am.keeper.EndBlocker(sdkCtx)
}

// HasInvariants: 불변성 검사 등록
func (am AppModule) RegisterInvariants(ir sdk.InvariantRegistry) {
    keeper.RegisterInvariants(ir, am.keeper)
}

// HasConsensusVersion: 모듈 버전
func (AppModule) ConsensusVersion() uint64 {
    return 2  // 모듈 버전 (상태 변경 시 증가)
}
```

Legacy (v0.47) vs New (v0.50+) 비교:

```go
// ❌ Legacy (v0.47)
func (am AppModule) BeginBlock(ctx sdk.Context, req abci.RequestBeginBlock) {
    // req.Header, req.LastCommitInfo 등 사용 가능
    height := req.Header.Height
}

// ✅ New (v0.50+)
func (am AppModule) BeginBlock(ctx context.Context) error {
    sdkCtx := sdk.UnwrapSDKContext(ctx)
    // Header 정보는 sdkCtx에서 가져오기
    height := sdkCtx.BlockHeight()
    return nil
}
```

Module Manager

ModuleManager는 여러 모듈을 관리하고 조율하는 컴포넌트입니다.

ModuleManager 구조

```go
// types/module/module.go
type Manager struct {
    Modules                 map[string]AppModule
    OrderInitGenesis        []string
    OrderExportGenesis      []string
    OrderBeginBlockers      []string
    OrderEndBlockers        []string
    OrderMigrations         []string
}
```

ModuleManager 생성

```go
// app.go
app.mm = module.NewManager(
    auth.NewAppModule(app.AccountKeeper),
    bank.NewAppModule(app.BankKeeper, app.AccountKeeper),
    staking.NewAppModule(app.StakingKeeper, app.AccountKeeper, app.BankKeeper),
    distribution.NewAppModule(app.DistrKeeper, app.AccountKeeper, app.BankKeeper, app.StakingKeeper),
    slashing.NewAppModule(app.SlashingKeeper, app.AccountKeeper, app.BankKeeper, app.StakingKeeper),
    gov.NewAppModule(app.GovKeeper, app.AccountKeeper, app.BankKeeper),
    mint.NewAppModule(app.MintKeeper, app.AccountKeeper),
    // ... 기타 모듈
)
```

실행 순서 설정

모듈의 실행 순서는 매우 중요합니다. 의존성이 있는 모듈은 순서를 고려해야 합니다.

InitGenesis 순서

```
app.mm.SetOrderInitGenesis(
    authtypes.ModuleName,       // 1. Account 먼저
    banktypes.ModuleName,       // 2. Balance 설정
    distrtypes.ModuleName,      // 3. Distribution 초기화
    stakingtypes.ModuleName,    // 4. Staking (validator 생성)
    slashingtypes.ModuleName,   // 5. Slashing
    govtypes.ModuleName,        // 6. Governance
    minttypes.ModuleName,       // 7. Mint
    // ...
)
```

왜 순서가 중요한가?

- **auth → bank**: Account가 존재해야 balance를 설정할 수 있음
- **bank → staking**: Delegation을 하려면 bank balance가 필요
- **staking → slashing**: Validator가 존재해야 slashing 정보를 설정

BeginBlockers 순서

```
app.mm.SetOrderBeginBlockers(
    minttypes.ModuleName,       // 1. 새 토큰 발행
    distrtypes.ModuleName,      // 2. 리워드 분배
    slashingtypes.ModuleName,   // 3. Validator slashing
    stakingtypes.ModuleName,    // 4. Staking 로직
    // ...
)
```

EndBlockers 순서

```
app.mm.SetOrderEndBlockers(
    govtypes.ModuleName,        // 1. Proposal 처리
    stakingtypes.ModuleName,    // 2. Validator set 업데이트
    // ...
)
```

ModuleManager 주요 메소드

InitGenesis

```go
func (m *Manager) InitGenesis(ctx sdk.Context, cdc codec.JSONCodec, genesisData map[string]json.RawMessage) abci.ResponseInitChain {
    var validatorUpdates []abci.ValidatorUpdate

    // 설정된 순서대로 각 모듈의 InitGenesis 호출
    for _, moduleName := range m.OrderInitGenesis {
        if genesisData[moduleName] == nil {
            continue
        }

        moduleValUpdates := m.Modules[moduleName].InitGenesis(ctx, cdc, genesisData[moduleName])
        if len(moduleValUpdates) > 0 {
            if len(validatorUpdates) > 0 {
                panic("validator InitGenesis updates already set by a previous module")
            }
            validatorUpdates = moduleValUpdates
        }
    }

    return abci.ResponseInitChain{
        Validators: validatorUpdates,
    }
}
```

BeginBlock

```go
func (m *Manager) BeginBlock(ctx sdk.Context, req abci.RequestBeginBlock) abci.ResponseBeginBlock {
    ctx = ctx.WithEventManager(sdk.NewEventManager())

    // 설정된 순서대로 각 모듈의 BeginBlock 호출
    for _, moduleName := range m.OrderBeginBlockers {
        if module, ok := m.Modules[moduleName].(BeginBlockAppModule); ok {
            module.BeginBlock(ctx, req)
        }
    }

    return abci.ResponseBeginBlock{
        Events: ctx.EventManager().ABCIEvents(),
    }
}
```

EndBlock

```go
func (m *Manager) EndBlock(ctx sdk.Context, req abci.RequestEndBlock) abci.ResponseEndBlock {
    ctx = ctx.WithEventManager(sdk.NewEventManager())
    validatorUpdates := []abci.ValidatorUpdate{}

    // 설정된 순서대로 각 모듈의 EndBlock 호출
    for _, moduleName := range m.OrderEndBlockers {
        if module, ok := m.Modules[moduleName].(EndBlockAppModule); ok {
            moduleValUpdates := module.EndBlock(ctx, req)
            if len(moduleValUpdates) > 0 {
                if len(validatorUpdates) > 0 {
                    panic("validator EndBlock updates already set by a previous module")
                }
                validatorUpdates = moduleValUpdates
            }
        }
    }

    return abci.ResponseEndBlock{
        ValidatorUpdates: validatorUpdates,
        Events:           ctx.EventManager().ABCIEvents(),
    }
}
```

Module 생명주기

1. 애플리케이션 시작 (InitChain)

```
Application Start
    ↓
ModuleManager.InitGenesis()
    ↓
For each module (in order):
    ↓
  Module.InitGenesis(ctx, genesisData)
    ↓
  - Load initial state
  - Set parameters
  - Initialize stores
```

예: Bank 모듈 InitGenesis

```go
// x/bank/keeper/genesis.go
func (k Keeper) InitGenesis(ctx sdk.Context, genState *types.GenesisState) {
    // 1. 파라미터 설정
    k.SetParams(ctx, genState.Params)

    // 2. Supply 초기화
    for _, supply := range genState.Supply {
        k.setSupply(ctx, supply)
    }

    // 3. 계정 잔액 설정
    for _, balance := range genState.Balances {
        addr, err := sdk.AccAddressFromBech32(balance.Address)
        if err != nil {
            panic(err)
        }

        for _, coin := range balance.Coins {
            k.setBalance(ctx, addr, coin)
        }
    }

    // 4. Denom metadata 설정
    for _, metadata := range genState.DenomMetadata {
        k.SetDenomMetaData(ctx, metadata)
    }
}
```

2. 블록 처리

```
New Block
    ↓
BeginBlock
    ↓
For each module (in order):
    ↓
  Module.BeginBlock(ctx, req)
    ↓
    ↓
For each transaction:
    ↓
  CheckTx or DeliverTx
    ↓
  Message routing to module
    ↓
  Module.MsgServer.Handle(ctx, msg)
    ↓
    ↓
EndBlock
    ↓
For each module (in order):
    ↓
  Module.EndBlock(ctx, req)
    ↓
Commit
```

3. BeginBlocker 예시

Distribution 모듈: 블록 리워드 분배

```go
// x/distribution/abci.go
func BeginBlocker(ctx sdk.Context, req abci.RequestBeginBlock, k keeper.Keeper) {
    // 이전 블록의 proposer에게 리워드 지급
    if ctx.BlockHeight() > 1 {
        previousProposer := k.GetPreviousProposerConsAddr(ctx)
        rewards := k.AllocateTokens(ctx, req.LastCommitInfo.Votes, previousProposer)

        // 이벤트 발생
        ctx.EventManager().EmitEvent(
            sdk.NewEvent(
                types.EventTypeProposerReward,
                sdk.NewAttribute(sdk.AttributeKeyAmount, rewards.String()),
                sdk.NewAttribute(types.AttributeKeyValidator, previousProposer.String()),
            ),
        )
    }

    // 현재 proposer 저장
    consAddr := sdk.ConsAddress(req.Header.ProposerAddress)
    k.SetPreviousProposerConsAddr(ctx, consAddr)
}
```

4. EndBlocker 예시

Staking 모듈: Validator set 업데이트

```go
// x/staking/abci.go
func EndBlocker(ctx sdk.Context, k keeper.Keeper) []abci.ValidatorUpdate {
    // 1. Unbonding 완료 처리
    k.CompleteUnbonding(ctx)
    k.CompleteRedelegation(ctx)

    // 2. Validator set 변경사항 계산
    return k.BlockValidatorUpdates(ctx)
}

func (k Keeper) BlockValidatorUpdates(ctx sdk.Context) []abci.ValidatorUpdate {
    // 변경사항을 추적하는 구조
    updates := []abci.ValidatorUpdate{}

    // Bonded validators를 순회하며 power 변경 확인
    k.IterateBondedValidatorsByPower(ctx, func(index int64, validator types.ValidatorI) (stop bool) {
        if validator.IsJailed() {
            // Jailed validator 제거
            updates = append(updates, validator.ABCIValidatorUpdateZero())
        } else if validator.IsBonded() {
            // Power 업데이트
            updates = append(updates, validator.ABCIValidatorUpdate())
        }
        return false
    })

    return updates
}
```

5. Genesis Export

```go
// x/bank/keeper/genesis.go
func (k Keeper) ExportGenesis(ctx sdk.Context) *types.GenesisState {
    return &types.GenesisState{
        Params:        k.GetParams(ctx),
        Balances:      k.GetAllBalances(ctx),
        Supply:        k.GetSupply(ctx),
        DenomMetadata: k.GetAllDenomMetaData(ctx),
    }
}
```

Keeper 패턴

Keeper는 모듈의 상태를 관리하는 핵심 컴포넌트입니다.

Keeper 구조

```go
// x/mymodule/keeper/keeper.go
type Keeper struct {
    // Store key
    storeKey sdk.StoreKey

    // Codec for binary encoding/decoding
    cdc codec.BinaryCodec

    // Parameter store (optional)
    paramSpace paramtypes.Subspace

    // 다른 모듈의 Keeper (의존성)
    accountKeeper types.AccountKeeper
    bankKeeper    types.BankKeeper

    // Authority (governance)
    authority string
}

func NewKeeper(
    cdc codec.BinaryCodec,
    storeKey sdk.StoreKey,
    paramSpace paramtypes.Subspace,
    accountKeeper types.AccountKeeper,
    bankKeeper types.BankKeeper,
    authority string,
) Keeper {
    // Param store 설정
    if !paramSpace.HasKeyTable() {
        paramSpace = paramSpace.WithKeyTable(types.ParamKeyTable())
    }

    return Keeper{
        cdc:           cdc,
        storeKey:      storeKey,
        paramSpace:    paramSpace,
        accountKeeper: accountKeeper,
        bankKeeper:    bankKeeper,
        authority:     authority,
    }
}
```

Store 접근

```go
// Get store from context
func (k Keeper) getStore(ctx sdk.Context) sdk.KVStore {
    return ctx.KVStore(k.storeKey)
}

// Set a value
func (k Keeper) SetItem(ctx sdk.Context, key string, value types.Item) {
    store := k.getStore(ctx)
    bz := k.cdc.MustMarshal(&value)
    store.Set(types.ItemKey(key), bz)
}

// Get a value
func (k Keeper) GetItem(ctx sdk.Context, key string) (types.Item, bool) {
    store := k.getStore(ctx)
    bz := store.Get(types.ItemKey(key))
    if bz == nil {
        return types.Item{}, false
    }

    var item types.Item
    k.cdc.MustUnmarshal(bz, &item)
    return item, true
}

// Delete a value
func (k Keeper) DeleteItem(ctx sdk.Context, key string) {
    store := k.getStore(ctx)
    store.Delete(types.ItemKey(key))
}
```

Iterator 패턴

```go
// Iterate over all items
func (k Keeper) IterateItems(ctx sdk.Context, cb func(key string, item types.Item) (stop bool)) {
    store := k.getStore(ctx)
    iterator := sdk.KVStorePrefixIterator(store, types.ItemPrefix)
    defer iterator.Close()

    for ; iterator.Valid(); iterator.Next() {
        var item types.Item
        k.cdc.MustUnmarshal(iterator.Value(), &item)

        key := string(iterator.Key()[len(types.ItemPrefix):])
        if cb(key, item) {
            break
        }
    }
}

// Get all items
func (k Keeper) GetAllItems(ctx sdk.Context) []types.Item {
    items := []types.Item{}
    k.IterateItems(ctx, func(key string, item types.Item) bool {
        items = append(items, item)
        return false
    })
    return items
}
```

Parameter 관리

```go
// Set parameters
func (k Keeper) SetParams(ctx sdk.Context, params types.Params) {
    k.paramSpace.SetParamSet(ctx, &params)
}

// Get parameters
func (k Keeper) GetParams(ctx sdk.Context) types.Params {
    var params types.Params
    k.paramSpace.GetParamSet(ctx, &params)
    return params
}

// Get a specific parameter
func (k Keeper) GetMaxItems(ctx sdk.Context) uint64 {
    var maxItems uint64
    k.paramSpace.Get(ctx, types.KeyMaxItems, &maxItems)
    return maxItems
}
```

Inter-Module Communication

모듈 간 통신은 Keeper 인터페이스를 통해 이루어집니다.

Expected Keepers

다른 모듈의 기능을 사용하기 위해 인터페이스를 정의합니다.

```go
// x/mymodule/types/expected_keepers.go
type AccountKeeper interface {
    GetAccount(ctx sdk.Context, addr sdk.AccAddress) authtypes.AccountI
    SetAccount(ctx sdk.Context, acc authtypes.AccountI)
    NewAccountWithAddress(ctx sdk.Context, addr sdk.AccAddress) authtypes.AccountI
}

type BankKeeper interface {
    SendCoins(ctx sdk.Context, fromAddr sdk.AccAddress, toAddr sdk.AccAddress, amt sdk.Coins) error
    GetBalance(ctx sdk.Context, addr sdk.AccAddress, denom string) sdk.Coin
    GetAllBalances(ctx sdk.Context, addr sdk.AccAddress) sdk.Coins
    SpendableCoins(ctx sdk.Context, addr sdk.AccAddress) sdk.Coins
}
```

Keeper 의존성 주입

```go
// app.go
app.MyModuleKeeper = mymodulekeeper.NewKeeper(
    appCodec,
    keys[mymoduletypes.StoreKey],
    app.GetSubspace(mymoduletypes.ModuleName),
    app.AccountKeeper,  // 의존성 주입
    app.BankKeeper,     // 의존성 주입
    authtypes.NewModuleAddress(govtypes.ModuleName).String(),
)
```

모듈 간 호출 예시

```go
// x/mymodule/keeper/msg_server.go
func (k msgServer) CreateItem(goCtx context.Context, msg *types.MsgCreateItem) (*types.MsgCreateItemResponse, error) {
    ctx := sdk.UnwrapSDKContext(goCtx)

    // 1. Account 확인 (auth 모듈)
    creator, err := sdk.AccAddressFromBech32(msg.Creator)
    if err != nil {
        return nil, err
    }
    account := k.accountKeeper.GetAccount(ctx, creator)
    if account == nil {
        return nil, sdkerrors.Wrapf(sdkerrors.ErrUnknownAddress, "account %s not found", msg.Creator)
    }

    // 2. Fee 차감 (bank 모듈)
    fee := sdk.NewCoins(sdk.NewInt64Coin("stake", 1000))
    if err := k.bankKeeper.SendCoinsFromAccountToModule(ctx, creator, types.ModuleName, fee); err != nil {
        return nil, err
    }

    // 3. Item 생성 (자체 로직)
    item := types.Item{
        Creator: msg.Creator,
        Name:    msg.Name,
        // ...
    }
    k.SetItem(ctx, msg.Name, item)

    // 4. 이벤트 발생
    ctx.EventManager().EmitEvent(
        sdk.NewEvent(
            types.EventTypeCreateItem,
            sdk.NewAttribute(types.AttributeKeyCreator, msg.Creator),
            sdk.NewAttribute(types.AttributeKeyItemName, msg.Name),
        ),
    )

    return &types.MsgCreateItemResponse{}, nil
}
```

Module Account

모듈이 자체적으로 토큰을 보유할 수 있습니다.

```go
// Module account 생성 (app.go)
maccPerms := map[string][]string{
    mymoduletypes.ModuleName: {authtypes.Minter, authtypes.Burner},
    // Minter: 토큰 발행 권한
    // Burner: 토큰 소각 권한
}

// Module account 사용
moduleAddr := k.accountKeeper.GetModuleAddress(types.ModuleName)
balance := k.bankKeeper.GetBalance(ctx, moduleAddr, "stake")
```

실전: Custom Module 개발

간단한 “게시판” 모듈을 개발해봅시다.

1. Proto 정의

```
// proto/mychain/board/v1/types.proto
syntax = "proto3";
package mychain.board.v1;

import "cosmos/base/v1beta1/coin.proto";
import "gogoproto/gogo.proto";

message Post {
  uint64 id = 1;
  string creator = 2;
  string title = 3;
  string content = 4;
  int64 created_at = 5;
  repeated cosmos.base.v1beta1.Coin tips = 6 [(gogoproto.nullable) = false];
}

message Params {
  uint64 max_title_length = 1;
  uint64 max_content_length = 2;
  string creation_fee_denom = 3;
  string creation_fee_amount = 4;
}
```

```
// proto/mychain/board/v1/tx.proto
syntax = "proto3";
package mychain.board.v1;

import "mychain/board/v1/types.proto";

service Msg {
  rpc CreatePost(MsgCreatePost) returns (MsgCreatePostResponse);
  rpc TipPost(MsgTipPost) returns (MsgTipPostResponse);
  rpc DeletePost(MsgDeletePost) returns (MsgDeletePostResponse);
}

message MsgCreatePost {
  string creator = 1;
  string title = 2;
  string content = 3;
}

message MsgCreatePostResponse {
  uint64 id = 1;
}

message MsgTipPost {
  string creator = 1;
  uint64 post_id = 2;
  cosmos.base.v1beta1.Coin amount = 3 [(gogoproto.nullable) = false];
}

message MsgTipPostResponse {}

message MsgDeletePost {
  string creator = 1;
  uint64 post_id = 2;
}

message MsgDeletePostResponse {}
```

```
// proto/mychain/board/v1/query.proto
syntax = "proto3";
package mychain.board.v1;

import "mychain/board/v1/types.proto";
import "cosmos/base/query/v1beta1/pagination.proto";

service Query {
  rpc Post(QueryPostRequest) returns (QueryPostResponse);
  rpc Posts(QueryPostsRequest) returns (QueryPostsResponse);
  rpc Params(QueryParamsRequest) returns (QueryParamsResponse);
}

message QueryPostRequest {
  uint64 id = 1;
}

message QueryPostResponse {
  Post post = 1 [(gogoproto.nullable) = false];
}

message QueryPostsRequest {
  cosmos.base.query.v1beta1.PageRequest pagination = 1;
}

message QueryPostsResponse {
  repeated Post posts = 1 [(gogoproto.nullable) = false];
  cosmos.base.query.v1beta1.PageResponse pagination = 2;
}

message QueryParamsRequest {}

message QueryParamsResponse {
  Params params = 1 [(gogoproto.nullable) = false];
}
```

2. Keeper 구현

```go
// x/board/keeper/keeper.go
type Keeper struct {
    cdc           codec.BinaryCodec
    storeKey      sdk.StoreKey
    paramSpace    paramtypes.Subspace
    accountKeeper types.AccountKeeper
    bankKeeper    types.BankKeeper
    authority     string
}

// 다음 Post ID 가져오기/증가
func (k Keeper) GetNextPostID(ctx sdk.Context) uint64 {
    store := ctx.KVStore(k.storeKey)
    bz := store.Get(types.KeyNextPostID)
    if bz == nil {
        return 1
    }
    return sdk.BigEndianToUint64(bz)
}

func (k Keeper) SetNextPostID(ctx sdk.Context, id uint64) {
    store := ctx.KVStore(k.storeKey)
    store.Set(types.KeyNextPostID, sdk.Uint64ToBigEndian(id))
}

// Post CRUD
func (k Keeper) SetPost(ctx sdk.Context, post types.Post) {
    store := ctx.KVStore(k.storeKey)
    bz := k.cdc.MustMarshal(&post)
    store.Set(types.PostKey(post.Id), bz)
}

func (k Keeper) GetPost(ctx sdk.Context, id uint64) (types.Post, bool) {
    store := ctx.KVStore(k.storeKey)
    bz := store.Get(types.PostKey(id))
    if bz == nil {
        return types.Post{}, false
    }

    var post types.Post
    k.cdc.MustUnmarshal(bz, &post)
    return post, true
}

func (k Keeper) DeletePost(ctx sdk.Context, id uint64) {
    store := ctx.KVStore(k.storeKey)
    store.Delete(types.PostKey(id))
}

func (k Keeper) GetAllPosts(ctx sdk.Context) []types.Post {
    store := ctx.KVStore(k.storeKey)
    iterator := sdk.KVStorePrefixIterator(store, types.KeyPrefixPost)
    defer iterator.Close()

    posts := []types.Post{}
    for ; iterator.Valid(); iterator.Next() {
        var post types.Post
        k.cdc.MustUnmarshal(iterator.Value(), &post)
        posts = append(posts, post)
    }
    return posts
}
```

3. Message Handler 구현

```go
// x/board/keeper/msg_server.go
type msgServer struct {
    Keeper
}

func NewMsgServerImpl(keeper Keeper) types.MsgServer {
    return &msgServer{Keeper: keeper}
}

func (k msgServer) CreatePost(goCtx context.Context, msg *types.MsgCreatePost) (*types.MsgCreatePostResponse, error) {
    ctx := sdk.UnwrapSDKContext(goCtx)

    // 1. 파라미터 검증
    params := k.GetParams(ctx)
    if uint64(len(msg.Title)) > params.MaxTitleLength {
        return nil, sdkerrors.Wrap(types.ErrTitleTooLong, fmt.Sprintf("max length: %d", params.MaxTitleLength))
    }
    if uint64(len(msg.Content)) > params.MaxContentLength {
        return nil, sdkerrors.Wrap(types.ErrContentTooLong, fmt.Sprintf("max length: %d", params.MaxContentLength))
    }

    // 2. Creator 주소 파싱
    creator, err := sdk.AccAddressFromBech32(msg.Creator)
    if err != nil {
        return nil, err
    }

    // 3. 생성 수수료 차감
    fee := sdk.NewCoins(sdk.NewCoin(params.CreationFeeDenom, sdk.NewIntFromString(params.CreationFeeAmount)))
    if err := k.bankKeeper.SendCoinsFromAccountToModule(ctx, creator, types.ModuleName, fee); err != nil {
        return nil, err
    }

    // 4. Post 생성
    id := k.GetNextPostID(ctx)
    post := types.Post{
        Id:        id,
        Creator:   msg.Creator,
        Title:     msg.Title,
        Content:   msg.Content,
        CreatedAt: ctx.BlockTime().Unix(),
        Tips:      sdk.NewCoins(),
    }
    k.SetPost(ctx, post)
    k.SetNextPostID(ctx, id+1)

    // 5. 이벤트 발생
    ctx.EventManager().EmitEvents(sdk.Events{
        sdk.NewEvent(
            types.EventTypeCreatePost,
            sdk.NewAttribute(types.AttributeKeyPostID, fmt.Sprintf("%d", id)),
            sdk.NewAttribute(types.AttributeKeyCreator, msg.Creator),
            sdk.NewAttribute(types.AttributeKeyTitle, msg.Title),
        ),
        sdk.NewEvent(
            sdk.EventTypeMessage,
            sdk.NewAttribute(sdk.AttributeKeyModule, types.ModuleName),
            sdk.NewAttribute(sdk.AttributeKeySender, msg.Creator),
        ),
    })

    return &types.MsgCreatePostResponse{Id: id}, nil
}

func (k msgServer) TipPost(goCtx context.Context, msg *types.MsgTipPost) (*types.MsgTipPostResponse, error) {
    ctx := sdk.UnwrapSDKContext(goCtx)

    // 1. Post 존재 확인
    post, found := k.GetPost(ctx, msg.PostId)
    if !found {
        return nil, sdkerrors.Wrapf(types.ErrPostNotFound, "post id: %d", msg.PostId)
    }

    // 2. Tipper 주소 파싱
    tipper, err := sdk.AccAddressFromBech32(msg.Creator)
    if err != nil {
```

```go
        return nil, err
    }

    // 3. Post creator 주소 파싱
    postCreator, err := sdk.AccAddressFromBech32(post.Creator)
    if err != nil {
        return nil, err
    }

    // 4. Tip 전송
    if err := k.bankKeeper.SendCoins(ctx, tipper, postCreator, sdk.NewCoins(msg.Amount)); err != nil {
        return nil, err
    }

    // 5. Post tips 업데이트
    post.Tips = post.Tips.Add(msg.Amount)
    k.SetPost(ctx, post)

    // 6. 이벤트 발생
    ctx.EventManager().EmitEvent(
        sdk.NewEvent(
            types.EventTypeTipPost,
            sdk.NewAttribute(types.AttributeKeyPostID, fmt.Sprintf("%d", msg.PostId)),
            sdk.NewAttribute(types.AttributeKeyTipper, msg.Creator),
            sdk.NewAttribute(sdk.AttributeKeyAmount, msg.Amount.String()),
        ),
    )

    return &types.MsgTipPostResponse{}, nil
}

func (k msgServer) DeletePost(goCtx context.Context, msg *types.MsgDeletePost) (*types.MsgDeletePostResponse, error) {
    ctx := sdk.UnwrapSDKContext(goCtx)

    // 1. Post 존재 확인
    post, found := k.GetPost(ctx, msg.PostId)
    if !found {
        return nil, sdkerrors.Wrapf(types.ErrPostNotFound, "post id: %d", msg.PostId)
    }

    // 2. Creator 확인 (본인만 삭제 가능)
    if post.Creator != msg.Creator {
        return nil, sdkerrors.Wrap(sdkerrors.ErrUnauthorized, "only post creator can delete")
    }

    // 3. Post 삭제
    k.DeletePost(ctx, msg.PostId)

    // 4. 이벤트 발생
    ctx.EventManager().EmitEvent(
        sdk.NewEvent(
            types.EventTypeDeletePost,
            sdk.NewAttribute(types.AttributeKeyPostID, fmt.Sprintf("%d", msg.PostId)),
            sdk.NewAttribute(types.AttributeKeyCreator, msg.Creator),
        ),
    )

    return &types.MsgDeletePostResponse{}, nil
}
```

4. Query Handler 구현

```go
// x/board/keeper/query_server.go
var _ types.QueryServer = Keeper{}

func (k Keeper) Post(goCtx context.Context, req *types.QueryPostRequest) (*types.QueryPostResponse, error) {
    if req == nil {
        return nil, status.Error(codes.InvalidArgument, "invalid request")
    }

    ctx := sdk.UnwrapSDKContext(goCtx)
    post, found := k.GetPost(ctx, req.Id)
    if !found {
        return nil, sdkerrors.Wrapf(types.ErrPostNotFound, "post id: %d", req.Id)
    }

    return &types.QueryPostResponse{Post: post}, nil
}

func (k Keeper) Posts(goCtx context.Context, req *types.QueryPostsRequest) (*types.QueryPostsResponse, error) {
    if req == nil {
        return nil, status.Error(codes.InvalidArgument, "invalid request")
    }

    ctx := sdk.UnwrapSDKContext(goCtx)
    store := ctx.KVStore(k.storeKey)
    postStore := prefix.NewStore(store, types.KeyPrefixPost)

    var posts []types.Post
    pageRes, err := query.Paginate(postStore, req.Pagination, func(key []byte, value []byte) error {
        var post types.Post
        if err := k.cdc.Unmarshal(value, &post); err != nil {
            return err
        }
        posts = append(posts, post)
        return nil
    })

    if err != nil {
        return nil, status.Error(codes.Internal, err.Error())
    }

    return &types.QueryPostsResponse{Posts: posts, Pagination: pageRes}, nil
}

func (k Keeper) Params(goCtx context.Context, req *types.QueryParamsRequest) (*types.QueryParamsResponse, error) {
    if req == nil {
        return nil, status.Error(codes.InvalidArgument, "invalid request")
    }

    ctx := sdk.UnwrapSDKContext(goCtx)
    params := k.GetParams(ctx)

    return &types.QueryParamsResponse{Params: params}, nil
}
```

5. Genesis 처리

```go
// x/board/keeper/genesis.go
func (k Keeper) InitGenesis(ctx sdk.Context, data types.GenesisState) {
    // 파라미터 설정
    k.SetParams(ctx, data.Params)

    // Post ID 초기화
    k.SetNextPostID(ctx, data.NextPostId)

    // Posts 로드
    for _, post := range data.Posts {
        k.SetPost(ctx, post)
    }
}

func (k Keeper) ExportGenesis(ctx sdk.Context) *types.GenesisState {
    return &types.GenesisState{
        Params:     k.GetParams(ctx),
        Posts:      k.GetAllPosts(ctx),
        NextPostId: k.GetNextPostID(ctx),
    }
}
```

Best Practices

1. Module 설계 원칙

✅ DO

- **단일 책임**: 하나의 모듈은 하나의 기능에 집중
- **명확한 인터페이스**: ExpectedKeepers를 통해 의존성 명시
- **상태 독립성**: 각 모듈의 state는 독립적으로 관리
- **이벤트 발생**: 중요한 상태 변경 시 이벤트 발생
- **파라미터화**: 하드코딩 대신 governance로 변경 가능한 파라미터 사용

❌ DON'T

- **순환 의존성**: A 모듈이 B를 의존하고, B가 다시 A를 의존
- **직접 Store 접근**: 다른 모듈의 store에 직접 접근
- **비결정론적 로직**: 시간, 난수 등 결정론적이지 않은 요소 사용
- **과도한 BeginBlock/EndBlock**: 블록 생성 시간에 영향을 주는 무거운 연산

2. Keeper 패턴

```go
// ✅ Good: Clear getter/setter pattern
func (k Keeper) SetBalance(ctx sdk.Context, addr sdk.AccAddress, coin sdk.Coin) {
    store := ctx.KVStore(k.storeKey)
    key := types.BalanceKey(addr, coin.Denom)
    bz := k.cdc.MustMarshal(&coin)
    store.Set(key, bz)
}

func (k Keeper) GetBalance(ctx sdk.Context, addr sdk.AccAddress, denom string) sdk.Coin {
    store := ctx.KVStore(k.storeKey)
    key := types.BalanceKey(addr, denom)
    bz := store.Get(key)
    if bz == nil {
        return sdk.NewCoin(denom, sdk.ZeroInt())
    }
    var coin sdk.Coin
    k.cdc.MustUnmarshal(bz, &coin)
    return coin
}

// ❌ Bad: Direct store manipulation in business logic
func (k Keeper) Transfer(ctx sdk.Context, from, to sdk.AccAddress, amount sdk.Coin) error {
    store := ctx.KVStore(k.storeKey)
    // ... direct store manipulation
    // 대신 GetBalance/SetBalance를 사용하세요
}
```

3. 에러 처리

```go
// ✅ Good: Custom errors with context
var (
    ErrPostNotFound  = sdkerrors.Register(ModuleName, 1, "post not found")
    ErrTitleTooLong  = sdkerrors.Register(ModuleName, 2, "title too long")
    ErrContentTooLong = sdkerrors.Register(ModuleName, 3, "content too long")
)

func (k Keeper) GetPost(ctx sdk.Context, id uint64) (types.Post, error) {
    post, found := k.getPost(ctx, id)
    if !found {
        return types.Post{}, sdkerrors.Wrapf(ErrPostNotFound, "post id: %d", id)
    }
    return post, nil
}
```

4. 이벤트 발생

```go
// ✅ Good: Descriptive events with attributes
ctx.EventManager().EmitEvents(sdk.Events{
    sdk.NewEvent(
        types.EventTypeCreatePost,
        sdk.NewAttribute(types.AttributeKeyPostID, fmt.Sprintf("%d", id)),
        sdk.NewAttribute(types.AttributeKeyCreator, msg.Creator),
        sdk.NewAttribute(types.AttributeKeyTitle, msg.Title),
    ),
    sdk.NewEvent(
        sdk.EventTypeMessage,
        sdk.NewAttribute(sdk.AttributeKeyModule, types.ModuleName),
        sdk.NewAttribute(sdk.AttributeKeySender, msg.Creator),
    ),
})
```

5. 테스트 작성

```go
// keeper_test.go
func (suite *KeeperTestSuite) TestCreatePost() {
    ctx := suite.ctx
    keeper := suite.keeper

    // Test data
    creator := sdk.AccAddress("creator")
    title := "Test Post"
    content := "This is a test post"

    // Setup
    suite.bankKeeper.EXPECT().
        SendCoinsFromAccountToModule(gomock.Any(), creator, types.ModuleName, gomock.Any()).
        Return(nil)

    // Execute
    id := keeper.GetNextPostID(ctx)
    post := types.Post{
        Id:        id,
        Creator:   creator.String(),
        Title:     title,
        Content:   content,
        CreatedAt: ctx.BlockTime().Unix(),
    }
    keeper.SetPost(ctx, post)

    // Verify
    savedPost, found := keeper.GetPost(ctx, id)
    suite.Require().True(found)
    suite.Require().Equal(post, savedPost)
}
```

참고 자료

공식 문서

- Cosmos SDK Modules
- Module Manager
- Keeper

소스 코드

- Bank 모듈
- Staking 모듈
- Module 타입 정의

예제 프로젝트

- Ignite CLI 템플릿
- Osmosis 모듈들

작성자: B-Harvest Core Developer Candidate **버전**: 1.0 **최종 수정일**: 2025-01-XX